

Lab 01 — The Interpreter, Numbers, Strings and Lists

DSA27 Lab Manual · Week 1 · Data Structures and Algorithms

Dr. Haitham A. El-Ghareeb

Fall 2026 · Week 1

Contents

Part 0 — Set up (before the lab)	2
Part 1 — Three ways to run Python	3
Part 2 — Numbers	4
Part 3 — Strings	8
Part 4 — Lists	13
Part 5 — First steps towards programming	15
Part 6 — Functions and tests, just enough to start	17
Part 7 — Exercises	19
Part 8 — Take-home practice (not graded)	20
Summary	21
Answers to the checkpoints	21

How to use this lab. Work through it from top to bottom with the Python interpreter open. Type every example yourself — do not copy and paste. When you see **Try it**, stop and do it. When you see **Checkpoint**, write down your prediction before you run the code. The answers are at the end. Then do the exercises in `Labs/Lab01.py` until `pytest tests/test_Lab01.py` passes.

Duration	One 2-hour lab session, plus about 2 hours at home
You will write	<code>labs/lab01.py</code> — 10 small functions
Graded by	<code>tests/test_lab01.py</code>
Before you come	Finish the setup in Part 0. The lab is not the time to install Python.

What you will be able to do

By the end of this lab you can:

1. start Python in three ways, and know when to use each;
2. use Python as a calculator, and say what `/`, `//`, `%` and `**` do — including with negative numbers;
3. make, index, slice and format strings, and use their common methods;
4. make, index, slice and change lists, and explain why `b = a` does **not** copy a list;
5. write a while loop, and a first `if`;
6. run a test file with `pytest` and read what it tells you.

Point 4 matters more than it looks. This entire course is about how data is arranged in memory, and Python lists are the first place you meet the idea that a name *refers to* an object rather than *containing* it.

Part 0 — Set up (before the lab)

You need **Python 3.10 or newer**. The course uses 3.13.

Windows. Install from <https://www.python.org/downloads/>. On the first screen of the installer tick **Add python.exe to PATH**. Then, in a new PowerShell window:

```
py --version
```

If python opens the Microsoft Store instead of running, turn off the Store aliases: *Settings* → *Apps* → *Advanced app settings* → *App execution aliases*, switch off `python.exe` and `python3.exe`.

macOS / Linux. Use `python3` wherever this manual says `py` or `python`.

Now get the course repository and create its virtual environment:

```
git clone https://github.com/helghareeb/DSA27.git
cd DSA27
py -3.13 -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
pytest -m "not challenge"
```

On macOS or Linux the two middle lines are `python3 -m venv .venv` and `source .venv/bin/activate`.

Every test in that last command must pass. They check your machine, not your code.

What is a virtual environment? A folder (`.venv`) holding a private copy of Python and its installed packages for *this project only*. `activate` makes the current terminal use it — your prompt changes to start with (`.venv`). Every time you open a new terminal to work on the course, `cd` into `DSA27` and activate again. Forgetting to activate is the most common reason a package “is installed but cannot be found”.

Editor. Use VS Code with the *Python* extension from Microsoft. Open the DSA27 folder (*File* → *Open Folder*), then choose the interpreter `.venv\Scripts\python.exe` when VS Code asks (or *Ctrl+Shift+P* → *Python: Select Interpreter*).

Part 1 — Three ways to run Python

1.1 The interactive interpreter (the REPL)

Type `python` (or `py`) in an activated terminal:

```
(.venv) PS C:\...\DSA27> python
Python 3.13.x ... on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

`>>>` is the **primary prompt**: Python is waiting for a statement. It **R**eads what you type, **E**valuates it, **P**rints the result, and **L**oops — hence *REPL*. When a statement continues over several lines you get the **secondary prompt** `...`:

```
>>> the_world_is_flat = True
>>> if the_world_is_flat:
...     print("Be careful not to fall off!")
...
Be careful not to fall off!
```

The empty `...` line (just press Enter) tells Python the block is finished.

To leave: type `exit()` or `quit()`, or press **Ctrl+Z then Enter** on Windows (**Ctrl+D** on macOS/Linux).

Two conveniences:

- In the REPL, the value of the last expression is stored in the variable `_` (underscore). Useful when you are using Python as a calculator.
- `help(len)` shows the documentation of anything. `help()` alone starts an interactive help system; `q` leaves a help page.

In this manual, code that starts with `>>>` is typed into the REPL. Lines with no prompt are what Python prints back.

1.2 Scripts

A script is a text file ending in `.py`. Create `hello.py` in the DSA27 folder:

```
# hello.py – my first script
name = "DSA27"
print("Hello,", name)
```

Run it from the terminal:

```
python hello.py
```

A script does **not** print the value of each expression the way the REPL does; only `print()` produces output. That surprises everyone once.

Everything after `#` on a line is a **comment** and is ignored. A `#` inside a string is just a character:

```
spam = 1          # a comment
text = "# this is not a comment"
```

Python source files are UTF-8, so Arabic works in strings and comments. Try `len('مرحبا')` in the REPL — it is 5, one per letter, not one per byte.

1.3 Notebooks

A Jupyter notebook mixes code cells, output and notes. The course has one per lecture in notebooks/. Start Jupyter with `jupyter lab`, or open an `.ipynb` file directly in VS Code and choose the `.venv` interpreter as the kernel.

Which one when? The REPL to try one line. A notebook to explore and draw. A `.py` file for anything you want to keep, test, or submit — which is every exercise in this course.

Try it. Start the REPL, type `import this`, and read The Zen of Python. (This is also part of Homework 1.) Then leave the REPL with `exit()`.

Part 2 — Numbers

2.1 Python as a calculator

```
>>> 2 + 2
4
>>> 50 - 5*6
20
>>> (50 - 5*6) / 4
5.0
>>> 8 / 5          # division ALWAYS gives a float
1.6
```

Numbers without a decimal point, like 2 and 20, are of type `int`. Numbers with one, like 5.0 and 1.6, are `float`. Ask Python with `type()`:

```
>>> type(20)
<class 'int'>
>>> type(5.0)
<class 'float'>
```

The arithmetic operators:

Operator	Meaning	Example	Result
+ - *	add, subtract, multiply	7 * 3	21
/	true division — always float	17 / 3	5.666666666666667
//	floor division — round down	17 // 3	5
%	remainder (modulo)	17 % 3	2
**	power	2 ** 10	1024

// and % go together: for any a and non-zero b,

```
a == (a // b) * b + a % b      # always True
```

```
>>> 17 // 3      # how many whole 3s fit in 17
5
>>> 17 % 3      # what is left over
2
>>> 5 * 3 + 2    # and back again
17
>>> divmod(17, 3) # both at once
(5, 2)
```

Floor means down, not towards zero. This matters with negative numbers and catches C and Java programmers:

```
>>> -7 // 2      # -3.5 rounded DOWN is -4
-4
>>> -7 % 2      # so the remainder must be 1: -4*2 + 1 == -7
1
```

Precedence. ** binds tighter than unary minus, and both bind tighter than * and /, which bind tighter than + and -. When unsure, use parentheses.

```
>>> -3 ** 2      # means -(3 ** 2)
-9
>>> (-3) ** 2
9
>>> 2 ** 3 ** 2  # ** groups right to left: 2 ** 9
512
```

2.2 Integers never overflow

In C an int is usually 32 bits and silently wraps around. A Python int grows as large as memory allows:

```
>>> 2 ** 100
1267650600228229401496703205376
>>> 2 ** 64 - 1      # the largest unsigned 64-bit integer in C
18446744073709551615
```

This is convenient, and it has a cost: arithmetic on huge integers is not $O(1)$. Adding two 1000-digit numbers takes longer than adding two small ones. Keep that in mind in Week 2.

2.3 Floats are approximations

A float is a 64-bit binary fraction. Most decimal fractions cannot be stored exactly:

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> round(0.1 + 0.2, 2) == 0.3
True
```

This is not a Python bug; C, Java and JavaScript do exactly the same. Two rules follow:

1. **Never compare floats with `==`** after arithmetic. Round, or check that the difference is tiny:
`abs(x - y) < 1e-9`.
2. **Never store money as a float.** Store piasters (or cents) as an `int`.

Mixing `int` and `float` gives a `float`:

```
>>> 4 * 3.75 - 1
14.0
```

`round(x, n)` rounds to n decimal places; `round(x)` gives the nearest `int`. Python rounds exact halves to the nearest **even** number ("banker's rounding"), so `round(2.5)` is 2 and `round(3.5)` is 4.

2.4 Variables and assignment

`=` binds a **name** to a value. It prints nothing.

```
>>> width = 20
>>> height = 5 * 9
>>> width * height
900
```

Using a name that was never assigned is an error:

```
>>> n
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'n' is not defined
```

Read error messages **from the bottom up**. The last line says what went wrong (NameError) and why (name 'n' is not defined). Lab 02 is about errors in detail.

Multiple assignment. The right-hand side is evaluated completely *first*, then the names on the left are bound left to right:

```
>>> a, b = 0, 1
>>> a, b = b, a + b      # uses the OLD a and b on the right
>>> a, b
(1, 1)
```

That is also the Python way to swap two variables — no temporary needed:

```
>>> x, y = 3, 7
>>> x, y = y, x
>>> x, y
(7, 3)
```

Augmented assignment. $x += 1$ means $x = x + 1$. Likewise $-=$, $*=$, $/=$, $//=$, $\%=$. Python has **no** $++$ or $--$ operator.

Names. Letters, digits and `_`, not starting with a digit, case-sensitive. Use `lower_case_with_underscores` for variables and functions — this is the Python convention (PEP 8), and the course code follows it.

2.5 Converting between types

```
>>> int("42") + 1
43
>>> float("3.5")
3.5
>>> int(3.99)          # int() truncates towards zero; it does not round
3
>>> int(-3.99)
-3
>>> str(42) + "!"
'42!'
>>> int("4.5")
Traceback (most recent call last):
...
ValueError: invalid literal for int() with base 10: '4.5'
```

`bool` is the type of `True` and `False`. They behave like 1 and 0 in arithmetic, which is occasionally handy: `True + True` is 2.

Checkpoint 1. Predict each result, then check it in the REPL.

- (a) `7 / 7` (b) `7 // 2` (c) `-7 // 2` (d) `7 % -2` (e) `2 ** -1`
 (f) `10 - 2 * 3 ** 2` (g) `int(" 12 ")` (h) `round(0.5) + round(1.5)`

Part 3 — Strings

3.1 Writing strings

A string (str) is a sequence of characters. Single and double quotes are equivalent; pick the one that saves you escaping:

```
>>> 'spam eggs'
'spam eggs'
>>> "doesn't"
"doesn't"
>>> 'doesn\'t'          # or escape the quote with a backslash
"doesn't"
>>> '"Yes," they said.'
'"Yes," they said.'
```

The REPL shows a string as *you would type it* (with quotes). `print()` shows the text itself:

```
>>> s = 'First line.\nSecond line.'      # \n is a newline
>>> s
'First line.\nSecond line.'
>>> print(s)
First line.
Second line.
```

Common escape sequences: `\n` newline, `\t` tab, `\\` a backslash, `\'` and `\"` quotes.

Raw strings switch escapes off. Use them for Windows paths and, later, regular expressions:

```
>>> print('C:\some\name')              # \n became a newline!
C:\some
ame
>>> print(r'C:\some\name')
C:\some\name
```

Python 3.12 and later also print `SyntaxWarning: invalid escape sequence '\s'` for the first line. That warning is a gift: it tells you a backslash in your string is not doing what you think. The fix is the `r` prefix.

Triple quotes (`"""..."""` or `'''...'''`) make a string that spans several lines. Every docstring in `dsa/` and `labs/` is one.

```
print("""\
Usage: thingy [OPTIONS]
      -h                Display this usage message
      -H hostname       Hostname to connect to
""")
```

The `\` at the end of the first line stops a newline being included there.

3.2 Joining and repeating

```
>>> 3 * 'un' + 'ium'
'ununinium'
>>> 'Py' 'thon'           # two literals side by side are joined
'Python'
>>> prefix = 'Py'
>>> prefix + 'thon'       # joining a variable needs +
'Python'
>>> 'Lab ' + 1
Traceback (most recent call last):
...
TypeError: can only concatenate str (not "int") to str
>>> 'Lab ' + str(1)
'Lab 1'
```

3.3 Indexing

Characters are numbered from **0**. Negative indices count from the end, starting at **-1**:

```
+---+---+---+---+---+---+
| P | y | t | h | o | n |
+---+---+---+---+---+---+
  0   1   2   3   4   5
-6  -5  -4  -3  -2  -1
```

```
>>> word = 'Python'
>>> word[0]
'P'
>>> word[5]
'n'
>>> word[-1]           # Last character
'n'
>>> word[-2]
'o'
>>> word[42]
Traceback (most recent call last):
...
IndexError: string index out of range
```

There is no separate character type: `word[0]` is simply a string of length 1.

3.4 Slicing

`s[i:j]` is the part from index `i` **up to but not including** `j`.

```

>>> word[0:2]
'Py'
>>> word[2:5]
'tho'
>>> word[:2]           # omitted start means 0
'Py'
>>> word[4:]           # omitted end means len(word)
'on'
>>> word[-2:]          # the last two characters
'on'
>>> word[:2] + word[2:] # always equal to word
'Python'

```

The best way to remember slices: the indices point **between** characters, as in the diagram above. `word[2:5]` is everything between edge 2 and edge 5.

The length of `s[i:j]` is `j - i` when both are in range. That “up to but not including” rule is why `range(n)` in Lab 02 stops at `n - 1`, and why so many algorithms in this course use *half-open* intervals `[lo, hi)`.

Out-of-range **indices** are errors, but out-of-range **slices** are quietly trimmed:

```

>>> word[4:42]
'on'
>>> word[42:]
''

```

A third number is the **step**:

```

>>> '0123456789'[:2] # every second character
'02468'
>>> word[::-1]       # step -1: the string reversed
'nohtyP'

```

3.5 Strings are immutable

You cannot change a string in place:

```

>>> word[0] = 'J'
Traceback (most recent call last):
...
TypeError: 'str' object does not support item assignment

```

Build a new one instead:

```

>>> 'J' + word[1:]
'Jython'

```

Every “changing” string method below returns a **new** string and leaves the original alone.

3.6 Useful string operations

```
>>> s = '  Data Structures  '
>>> len(s)
19
>>> s.strip()                # remove whitespace at both ends
'Data Structures'
>>> s.strip().lower()
'data structures'
>>> s.upper()
'  DATA STRUCTURES  '
>>> 'Structures' in s        # substring test
True
>>> s.strip().split()        # split on runs of whitespace
['Data', 'Structures']
>>> 'a,b,,c'.split(',')      # split on an exact separator
['a', 'b', '', 'c']
>>> '-'.join(['2026', '09', '23'])
'2026-09-23'
>>> 'banana'.count('a')
3
>>> 'banana'.find('n')       # first index, or -1 if absent
2
>>> 'banana'.index('n')     # first index, or ValueError if absent
2
>>> 'banana'.replace('a', 'o')
'bonono'
>>> 'lab01.py'.endswith('.py')
True
>>> '42'.isdigit()
True
>>> 'x..!'.strip('!.')      # strip the given characters instead
'x'
```

You do not need to memorise these. You need to know they exist, and how to find them: `help(str)` or `dir(str)`.

3.7 Printing and f-strings

`print()` takes any number of arguments, separates them with a space, and ends with a newline. Both can be changed:

```
>>> print('a', 'b', 'c')
a b c
>>> print('a', 'b', 'c', sep='-')
a-b-c
```

```
>>> print('no newline', end=' | ')
no newline | >>>
```

An **f-string** — a string with `f` before the quote — evaluates expressions in `{ }` and inserts them:

```
>>> name, n = 'Ada', 3
>>> f'{name} solved {n} problems'
'Ada solved 3 problems'
>>> f'{n * 2 + 1}'
'7'
```

After a `:` you can say how to format the value:

```
>>> pi = 3.14159265
>>> f'{pi:.2f}'           # 2 digits after the decimal point
'3.14'
>>> f'{7:03d}'           # an int, at least 3 wide, padded with zeros
'007'
>>> f'[{ "hi":>6}]'       # right-align in 6 characters
'[    hi]'
>>> f'[{ "hi":<6}]'       # left-align
'[hi    ]'
>>> f'{1234567:,}'        # thousands separator
'1,234,567'
>>> f'{n = }'             # show the expression and its value – great for debugging
'n = 3'
```

A small table, with columns lined up:

```
>>> for name, mark in [('Ali', 91.5), ('Mona', 88), ('Yousef', 100)]:
...     print(f'{name:<8}{mark:>7.1f}')
...
Ali          91.5
Mona         88.0
Yousef       100.0
```

(for is in Lab 02. Here just enjoy the alignment.)

3.8 Reading input

`input(prompt)` prints the prompt, waits for a line of text and returns it — always as a **string**:

```
>>> age = input('Age? ')
Age? 20
>>> age + 1
Traceback (most recent call last):
...
TypeError: can only concatenate str (not "int") to str
>>> int(age) + 1
```

21

Exercises in this course never use `input()`: a function that asks the keyboard cannot be tested automatically. Functions take **parameters** and **return** results. Keep `input()` for small scripts you run by hand.

Checkpoint 2. With `s = 'Mansoura'`, predict:

- (a) `s[1]` (b) `s[-3:]` (c) `s[2:5]` (d) `s[:3]` (e) `s[5:2]`
 (f) `s.lower().count('a')` (g) `s[:3] * 2` (h) `f'{s:*^12}'`

Part 4 — Lists

4.1 Making and reading lists

A list is an **ordered, changeable** sequence of values, written in square brackets:

```
>>> squares = [1, 4, 9, 16, 25]
>>> squares[0]
1
>>> squares[-1]
25
>>> squares[-3:]          # slicing works exactly as for strings
[9, 16, 25]
>>> len(squares)
5
>>> squares + [36, 49]    # + makes a NEW List
[1, 4, 9, 16, 25, 36, 49]
>>> [0] * 4
[0, 0, 0, 0]
```

A list can hold values of different types, including other lists:

```
>>> mixed = [42, 'hello', 3.14, [1, 2]]
>>> mixed[3][0]          # first element of the inner list
1
```

4.2 Lists are mutable

Unlike strings, lists can be changed in place:

```
>>> cubes = [1, 8, 27, 65, 125]    # something is wrong here
>>> 4 ** 3
64
>>> cubes[3] = 64                  # replace the wrong value
>>> cubes
[1, 8, 27, 64, 125]
```

```
>>> cubes.append(216)           # add to the end
>>> cubes.append(7 ** 3)
>>> cubes
[1, 8, 27, 64, 125, 216, 343]
```

You can even assign to a **slice**, which can change the length of the list:

```
>>> letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> letters[2:5] = ['C', 'D', 'E']    # replace some
>>> letters
['a', 'b', 'C', 'D', 'E', 'f', 'g']
>>> letters[2:5] = []                # remove them
>>> letters
['a', 'b', 'f', 'g']
>>> letters[:] = []                  # empty the whole list
>>> letters
[]
```

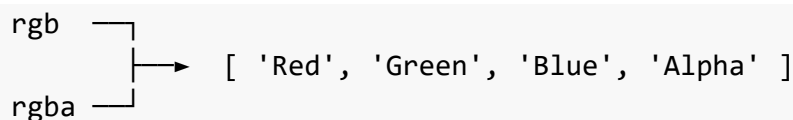
Lab 03 covers all the list methods (insert, pop, remove, sort, ...) and what each one costs.

4.3 Names refer to objects — the most important idea in this lab

This is the idea from Lecture 01: a Python variable is **a name bound to an object**, not a box holding a value. Assignment never copies.

```
>>> rgb = ['Red', 'Green', 'Blue']
>>> rgba = rgb                     # a second NAME for the SAME list
>>> rgba.append('Alpha')
>>> rgb                            # rgb "changed" too
['Red', 'Green', 'Blue', 'Alpha']
>>> rgba is rgb                    # `is` asks: the same object?
True
>>> id(rgb) == id(rgba)           # id() is the object's identity
True
```

Draw it:



To get a **separate** list, copy it. A full slice `[ : ]` returns a new list:

```
>>> correct_rgba = rgb[:]          # or rgb.copy() or list(rgb)
>>> correct_rgba.append('X')
>>> rgb
['Red', 'Green', 'Blue', 'Alpha']
>>> correct_rgba is rgb
False
```

```
>>> correct_rgba == rgb[:4] + ['X']    # == asks: equal VALUES?
True
```

`==` compares **values**; `is` compares **identity**. Two different lists can be `==` without being `is`.

The copy is shallow. `rgb[:]` is a new outer list, but it holds references to the *same* inner objects. With nested lists this bites:

```
>>> grid = [[0, 0], [0, 0]]
>>> copy = grid[:]
>>> copy[0][0] = 9
>>> grid
[[9, 0], [0, 0]]
```

And the classic trap — making a grid with `*`:

```
>>> board = [[0] * 3] * 3           # three references to ONE inner list
>>> board[0][0] = 'X'
>>> board
[['X', 0, 0], ['X', 0, 0], ['X', 0, 0]]
```

Lab 03 shows the right way (a list comprehension).

Why this matters for the course: when you build a linked list in Week 5, each node's `next` is exactly this kind of reference. If `b = a` surprises you now, `node.next = other` will surprise you then.

Checkpoint 3. Predict the final value of *a*, *b* and *c*.

```
a = [1, 2, 3]
b = a
c = a[:]
b[0] = 100
c[1] = 200
a = a + [4]
b.append(5)
```

Part 5 — First steps towards programming

5.1 A while loop

Here is the first few Fibonacci numbers, where each is the sum of the two before:

```
>>> a, b = 0, 1
>>> while a < 10:
...     print(a)
...     a, b = b, a + b
... 
```

```
0
1
1
2
3
5
8
```

Several ideas at once:

- **Multiple assignment** on the first and last lines, as in Part 2.4. The last line is the whole algorithm: *the new a is the old b; the new b is the old sum*.
- The **while loop** runs its body as long as the condition `a < 10` is true. In Python, any non-zero number and any non-empty string or list count as true; `0`, `''` and `[]` count as false.
- **Comparisons**: `<`, `>`, `==` (equal), `!=` (not equal), `<=`, `>=`.
- **Indentation is syntax**. The body of the loop is the indented lines. There are no braces. Use **4 spaces** per level — VS Code does this when you press Tab. Mixing tabs and spaces is an error.

`print(a, end=', ')` keeps the output on one line:

```
>>> a, b = 0, 1
>>> while a < 1000:
...     print(a, end=', ')
...     a, b = b, a + b
...
0,1,1,2,3,5,8,13,21,34,55,89,144,233,377,610,987,
```

5.2 A first if

`if` runs a block only when a condition is true. `else` runs otherwise. Lab 02 covers it properly; for now this is enough:

```
>>> n = 7
>>> if n % 2 == 0:
...     print('even')
... else:
...     print('odd')
...
odd
```

Combine conditions with `and`, `or` and `not`:

```
>>> x = 15
>>> x > 10 and x % 5 == 0
True
```

5.3 A loop that builds a list

The pattern you will write most often: start empty, append in a loop.

```
>>> powers = []
>>> p = 1
>>> while p < 100:
...     powers.append(p)
...     p = p * 2
...
>>> powers
[1, 2, 4, 8, 16, 32, 64]
```

And the same with an index, walking through an existing list:

```
>>> names = ['ali', 'mona', 'omar']
>>> i = 0
>>> while i < len(names):
...     print(i, names[i].upper())
...     i += 1
...
0 ALI
1 MONA
2 OMAR
```

Forget the `i += 1` and the loop never ends. Press **Ctrl+C** to stop a runaway program — you will need it at least once this term.

Checkpoint 4. *How many times does the body of this loop run, and what is printed?*

```
n, steps = 100, 0
while n > 1:
    n = n // 2
    steps += 1
print(n, steps)
```

This loop halves its input every time. Remember the count — it is the reason binary search, in Week 8, is so fast.

Part 6 — Functions and tests, just enough to start

The exercises are written as **functions**. Lab 02 teaches functions properly; here is the minimum.

```
def area(width, height):
    """Area of a rectangle."""
    return width * height
```

- `def` starts a function definition. `width` and `height` are **parameters**.

- The indented lines are the body.
- The string right after the `def` line is the **docstring** — the function's documentation and, in this course, its contract.
- `return` sends a value back to the caller and ends the function.

```
>>> area(3, 4)
12
>>> result = area(2, 5) + 1
>>> result
11
```

A function can return several values separated by commas. The caller gets them together as a **tuple** (more in Lab 03) and can unpack them:

```
def min_max(a, b):
    if a < b:
        return a, b
    return b, a

>>> min_max(9, 2)
(2, 9)
>>> low, high = min_max(9, 2)
```

print is not return. A function that prints its answer and returns nothing returns `None`, and every test will fail:

```
def bad_area(width, height):
    print(width * height)    # shows 12 on the screen ...

>>> x = bad_area(3, 4)
12
>>> print(x)                # ... but gives nothing back
None
```

6.1 How the exercises work

Open `labs/lab01.py`. Every function looks like this:

```
def celsius_to_fahrenheit(celsius):
    """Convert Celsius to Fahrenheit, rounded to one decimal place.
    ...
    celsius_to_fahrenheit(100)  -> 212.0
    """
    raise NotImplementedError
```

Replace `raise NotImplementedError` with your code. Then run the tests from the DSA27 folder, with the virtual environment active:

```
pytest tests/test_lab01.py -v      # every Lab 01 test, one per line
pytest tests/test_lab01.py -x      # stop at the first failure
pytest tests/test_lab01.py -k fib  # only tests with "fib" in their name
```

A failing test looks like this:

```
def test_celsius_to_fahrenheit(c, f):
>     assert celsius_to_fahrenheit(c) == f
E     assert 97.88000000000001 == 97.9
```

Read it as: *the test called your function with $c = 36.6$; you returned 97.88000000000001 ; it expected 97.9 . You forgot to round. The test file is the specification — open `tests/test_lab01.py` and read it whenever you are unsure what is wanted.*

You can also try a function by hand in the REPL:

```
>>> from labs.lab01 import celsius_to_fahrenheit
>>> celsius_to_fahrenheit(100)
212.0
```

After you change the file, the REPL still has the **old** version. Leave with `exit()` and start again, or use the tests — they always load the current file.

Part 7 — Exercises

All in `labs/lab01.py`. Do them in order. Each docstring gives examples. Use only what this lab taught: no `import`, no `for` (that is next week).

#	Function	Practises
1	<code>seconds_to_hms(seconds)</code>	<code>//</code> , <code>%</code> , f-string padding
2	<code>celsius_to_fahrenheit(celsius)</code>	float arithmetic, round
3	<code>split_evenly(total, people)</code>	<code>//</code> and <code>%</code> together, returning two values
4	<code>initials(full_name)</code>	<code>split</code> , indexing, <code>while</code> , building a string
5	<code>is_palindrome_word(word)</code>	<code>strip</code> , <code>lower</code> , slicing with <code>step -1</code>
6	<code>mask_email(email)</code>	index, slicing, string repetition, <code>if</code>
7	<code>middle(values)</code>	slice arithmetic with <code>//</code>
8	<code>rotate_left(values, k)</code>	slicing, <code>%</code> , not changing the input
9	<code>fib_list(n)</code>	<code>while</code> , multiple assignment, <code>append</code>
10	<code>collatz_steps(n)</code>	<code>while</code> , <code>if/else</code> , counting

Hints, only if you are stuck for more than ten minutes:

- **1.** Hours are seconds // 3600. What is left is seconds % 3600; do the same again with 60. Then `f'{h}:{m:02d}:{s:02d}'`.
- **4.** `' a b '.split()` is `['a', 'b']` — the spaces problem disappears. Start with `result = ''` and add `word[0].upper() + '.'` for each word.
- **6.** `at = email.index('@')`. The name is `email[:at]`; the domain, including the @, is `email[at:]`.
- **7.** Work out by hand which indices you want for lengths 0 to 5, and look for the pattern. `(n - 1) // 2` and `n // 2` are worth trying.
- **8.** An empty list needs its own `if`, because `k % 0` is an error. Otherwise, after `k = k % len(values)`, the answer is two slices joined.
- **9.** Stop when the list has `n` numbers: `while len(result) < n:`.

When all ten pass:

```
tests/test_lab01.py ..... [100%]
```

show the TA.

Part 8 — Take-home practice (not graded)

Do these in a scratch file, for example `practice/week01.py`.

1. Write a script that prints the multiplication table from 1×1 to 9×9 in nine right-aligned rows, using two `while` loops and an f-string.
2. Given `s = 'Data Structures and Algorithms'`, print, each with one expression: the number of words; the string with its words in reverse order; the acronym `DSaA`; the string with every vowel replaced by `*`.
3. Without running it, write down what `x` is after `x = [1, [2, 3]]`; `y = x[:]`; `y[1].append(4)`; `y[0] = 9`. Then run it. Explain the result in one sentence using the words *shallow copy*.
4. Write `digits(n)` that returns the list of decimal digits of a non-negative integer, most significant first, using only `%`, `//` and a `while` loop. Why does `digits(0)` need thought?
5. Time something. In a script:

```
import time
start = time.perf_counter()
total, i = 0, 0
while i < 10_000_000:
    total += i
    i += 1
print(time.perf_counter() - start, 'seconds')
```

Run it with 1, 2, 4 and 8 million. What happens to the time when the input doubles? That question is all of Week 2.

Summary

Idea	The one line to keep
REPL vs script	The REPL prints every value; a script prints only what you <code>print</code> .
<code>/</code> vs <code>//</code> vs <code>%</code>	<code>/</code> is always float; <code>//</code> rounds down ; <code>a == (a//b)*b + a%b</code> .
<code>int</code> vs <code>float</code>	<code>int</code> never overflows; <code>float</code> is approximate — never <code>==</code> after arithmetic.
Strings	Immutable. Index from 0; -1 is last; <code>s[i:j]</code> stops <i>before</i> <code>j</code> .
Lists	Mutable. Same indexing and slicing as strings.
Names	A name refers to an object. <code>b = a</code> copies nothing . <code>a[:]</code> is a shallow copy.
<code>==</code> vs <code>is</code>	Equal values vs the same object.
Indentation	Is syntax. Four spaces.
<code>print</code> vs <code>return</code>	Tests see only what you <code>return</code> .

Answers to the checkpoints

Checkpoint 1. (a) `1.0` — `/` always gives a float. (b) 3. (c) -4 — floor rounds *down*. (d) -1 — the remainder takes the sign of the divisor: `7 == (-4)*(-2) + (-1)`. (e) `0.5`. (f) -8 — `3 ** 2` first, then `2 * 9`, then `10 - 18`. (g) 12 — `int()` ignores surrounding whitespace. (h) 2 — `round(0.5)` is 0 and `round(1.5)` is 2: halves go to the even neighbour.

Checkpoint 2. (a) `'a'` (b) `'ura'` (c) `'nso'` (d) `'Msr'` — indices 0, 3, 6 (e) `''` — the start is after the end, so the slice is empty (f) 2 (g) `'ManMan'` (h) `'**Mansoura**'` — centred (^) in 12 characters, padded with `*`.

Checkpoint 3. `a` is `[100, 2, 3, 4]`, `b` is `[100, 2, 3, 5]`, `c` is `[1, 200, 3]`. `b[0] = 100` changes the one list both `a` and `b` name. `c` is a separate copy. Then `a = a + [4]` builds a **new** list and rebinds `a` to it — `b` still names the old one, so `b.append(5)` no longer affects `a`. Compare this with `a += [4]`, which *extends the existing list in place* and would have changed `b` too.

Checkpoint 4. It prints 1 6. `n` goes 100, 50, 25, 12, 6, 3, 1 — six halvings. Doubling `n` to 200 adds only one more step. The number of halvings from `n` down to 1 is about $\log_2 n$.